

3. Documentation Review: I was provided with essential documentation, including team processes, best practices, and architectural overviews. This helped me understand the team's approach to the SRE foundation and the systems we manage.

4. Hands-on Training: The team organized hands-on training sessions covering key technologies and practices, such as Kubernetes, hashicorp vault, Yubikey access management, etc.

5. Mentorship Program: I was assigned a mentor who guided me throughout the internship, answering questions and helping me navigate challenges.

6. Gradual Responsibility: The team adopted a gradual approach to assigning responsibilities, starting with smaller tasks and progressively helping me to onboard our data platform infrastructure to the config-mgmt and helm chart repository.

7. Regular Check-ins: Weekly check-ins with my mentor and the SRE foundation team manager ensured I was progressing well and had the needed support.

This structured onboarding process significantly eased my transition into the SRE team and set a strong foundation for my internship experience.

Key Learnings

Throughout my internship, I gained numerous insights that have shaped my understanding of SRE practices:

Lesson learned - Technical

Learning new technologies overview Chef, Cloudflare, sentry, helm, and Terraform.

How are infrastructure, such as Code (IaC) principles, extensively used to manage and provision resources?

GitLab CI/CD pipelines are leveraged for automated deployments and testing.

Kubernetes is used for container orchestration and management.

Monitoring and alerting systems are in place to ensure high availability and quick response to issues.

Security measures, including encryption and access controls, are integrated into the infrastructure design.

The Hashicorp vault is configured to manage secrets securely across the infrastructure. It provides:

- Centralized secret management

- Dynamic secrets generation

- Encryption as a service

- Audit logging

- Access control policies

Key benefits of using Vault at GitLab:

- Enhanced security: Secrets are encrypted at rest and in transit

- Fine-grained access control: Policies define who can access which secrets

- Integration with CI/CD: Seamless integration with GitLab CI/CD pipelines

Auditing: Comprehensive logs for all secret access and operations

The SRE team manages Vault clusters for different environments (development, staging, production) and ensures high availability and disaster recovery capabilities.

Learned to host a new GCP project from scratch into the config-mgmt repository.
Cookbook setup in terms of permission and access control.

Lesson learned - Process

Over here, things were almost the same as our data teams regarding quarter planning, execution, and retro.

They have a few more calls than us as the team is geo-located, because of which 2 sync calls are weekly.

Fire drill call where they practise the production incident.

S1 sync call in case they have an open S1.

To review an MR, you don't need to assign it to any one particular person; people post any MR review request in sproductionengineering or gfoundations slack channel.

Everyone is a code owner, but you can't merge your changes. Also, one contributed to the MR and can't merge or approve the changes.

Everyone in the team uses Yubikey to connect to the production instance, ensuring the best security.

How does an internship with the SRE foundation benefit the data platform team?

With my internship kicked in, I got everyone in the data platform team to have Yubikey and a defined process to access the .com Postgres instance accessible from a local machine. A self-serving process is set up for any new onboarding team member.

I onboarded the whole data platform team on Hashicorp vault, which will help the team manage the kubernetes secrets for airflow instances running in production and testing namespace.

Together with Rigerta, we onboarded data platform staging airflow GKE cluster on the SRE team config-mgmt repo. (This ensures data platform infrastructure is highly reliable and secure, and we get controlled upgrades from Google). Learning from this is to roll out the production cluster to the config-mgmt.

There is knowledge in the team on which team in infrastructure is responsible for managing which infrastructure component.

Key takeaways from the SRE Foundation team internship

1. **Importance of Observability:** I learned that having comprehensive visibility into our systems is crucial for maintaining reliability. Proper instrumentation, logging, and monitoring are essential for quickly identifying, and resolving issues.

2. **Embracing Failure:** The SRE mindset of treating failures as learning opportunities rather than blame-worthy events was eye-opening. This approach fosters a culture of continuous improvement and innovation.

3. **Automation is Key:** I saw firsthand how automation can significantly reduce human error, increase efficiency, and allow the team to focus on more strategic initiatives.

4. **Cross-functional Collaboration:** SRE work involves close collaboration with development teams and other stakeholders. Effective communication and teamwork are essential for success.

5. **Continuous Learning:** The fast-paced nature of technology means that SREs must constantly update their skills and knowledge. I learned the importance of staying curious and adaptable in this field.

Challenges and Growth

While the internship was enriching, it also came with its share of challenges:

1. **Steep Learning Curve:** The sheer amount of new technologies, tools, and concepts I had to grasp was initially overwhelming. However, this challenge pushed me to develop effective learning strategies and time management skills.

2. **Dealing with Ambiguity:** Many problems in SRE don't have clear-cut solutions. Learning to navigate ambiguity and make decisions with incomplete information was a valuable skill I developed.

3. **Balancing Reliability and Innovation:** I learned the delicate balance between maintaining system stability and pushing for innovation and improvements.

How does an intern contribute?

Handbook update of SRE team.

Process sharing in terms of how we do weekly sync and what different topics we cover

An intern helps them update the document to make things self-serving, that is to prepare the runbook.

Performing review of small changes.

Key Takeaways for Future Interns

If you're considering an SRE internship, here are a few tips based on my experience:

Be Proactive: Don't wait for someone to tell you what to do. Seek out opportunities to learn and contribute.

Ask Questions: No question is too dumb. The more you ask, the more you'll learn.

Don't Be Afraid to Make Mistakes: Everyone makes mistakes. The important thing is to learn from them.

Network: Get to know your team members and others in the company. You never know where those connections might lead.

Have Fun! SRE can be challenging but incredibly rewarding.

Conclusion

My SRE internship has been an incredibly enriching experience that has deepened my understanding of Site Reliability Engineering and solidified my passion for this field. The skills, knowledge, and insights I've gained during this time have significantly enhanced my capabilities as a data engineer.

Throughout this journey, I've had the opportunity to work with cutting-edge technologies, collaborate with talented professionals, and contribute to maintaining and improving the reliability of large-scale systems. The hands-on experience in areas such as infrastructure as code, Kubernetes, GitLab CI/CD, and Hashicorp Vault has been invaluable.

I'm particularly grateful for the supportive and collaborative environment fostered by the SRE foundation team at GitLab. Their mentorship, guidance, and willingness to share knowledge have been instrumental in my growth. The culture of continuous learning, embracing Failure as a learning opportunity, and striving for constant improvement has left a lasting impression on me.

Moreover, this internship has highlighted the critical role that SRE plays in modern software development, and operations. The focus on observability, automation, and cross-functional collaboration has shown me how SRE practices can significantly enhance complex systems' reliability, scalability, and efficiency.

I'm excited about the future possibilities in the field of SRE, and look forward to applying the knowledge, and skills I've gained in my data engineering workflow. This internship has been a crucial stepping stone in my professional journey, and I'm grateful for the opportunity to have been part of such an innovative and impactful team.

Acknowledgements

I want to thank my mentors, team members, and the organization for providing me with this opportunity, and for their unwavering support throughout my internship.

Feel free to contact me if you have any questions about my experience or SRE. I'm always happy to connect, and share more about this exciting field!

Authored By

Ved Prakash

SECTION: enterprise-data/organization/programs/_index.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

<i id="biz-tech-icons" class="far fa-paper-plane"></i>Introduction

Welcome to the Data Programs page. Here you'll find information about the various Data Programs around GitLab and those the Data Team supports, ranging from onboarding to day-to-day operations.

- Data Slack Channels
- Primary Data Slack Channel: data
- Data Lounge Channel: data-lounge

<i class="far fa-newspaper" id="biz-tech-icons"></i> Show-n-Tell and Demos

Recordings of previous demos are posted to the GitLab Unfiltered Data Team playlist.

Data Onboarding

If you are onboarding to GitLab and will be working in the Data Program as an Engineer, Analyst or Developer, follow these steps:

1. Open a new issue in GitLab Data Analytics with the Data Onboarding template.
1. Give the issue a descriptive name: Your Name - Data Onboarding
1. Assign the issue to your Manager to add/remove relevant content

Data Proof of Value Guide

The Data Team performs Proof of Value Evaluations (PoVs) for all new technologies we are considering adding to the Data Platform or the broader Technology stack. This Guide is intended to help you perform a PoV efficiently and with great results.

Phase 1: Calculate Value and Define Requirements

1. Establish the Value the technology can provide GitLab. Value can be measured in a variety of ways, ranging from efficiency to increased Sales to reduced compute.
1. Create a Requirements document to define the business and technical requirements the technology must meet to be successful. Indicate whether each requirement is Must Have or Nice to Have. Here is a template we have used for Data Visualization PoV and another we have used for Product Analytics PoV.

Phase 2: Scoping & Policy Awareness

1. Review the Procurement New Software Guide to ensure you understand the latest procurement process to follow.
1. Execute an NDA with each Vendor included in the Evaluation.
1. Obtain preliminary pricing to help validate established budget. If no existing budget is established, work with the Department lead to determine if the project is feasible. Let's not waste time or energy for projects we can't fund.

Phase 3: Evaluation Design

Evaluation Design is the most complex part of the PoV.

1. Decide how to test the technology versus defined requirements. Often, successfully testing data technologies requires simulating production workloads and constructing a 'Production Level SAFE Workload' is a key challenge in a Data PoV Project.
 - No Red Data or Orange Data is ever suitable for inclusion in a PoV.

Phase 4: Procurement

1. Using the Requirements design as a guide, collaborate with the Vendor to create a Statement of Work (SoW).
 - The Statement of Work should include Success Criteria, Expectations, and a Project Timeline

- We do not pay for PoVs and all should be \$0 Cost
- 1. Along with the SoW, ask the vendor to send you their Master Services Agreement (MSA).
- 1. Because request with an amount of \$0 is not supported in Coupa, you need to submit the SoW and MSA to Procurement via GitLab in the Procurement project.
 - Submit the SoW and MSA to Procurement via Coupa if the PoV amount is not \$0, following this [How Do I Submit a Request to Purchase New Software?](#) guide.
 - The process to obtain security approvals is supported in Coupa.

Phase 5: Assessment

1. Create a shared Slack Channel to coordinate the PoV with the Vendor.
1. Reach out to the vendor for references to schedule a reference Call. In a reference call you can:
 - ask about the experience with the technology.
 - ask about their lessons learned.
 - ask how we can setup for success.

Phase 6: Wrap-Up

To Be Defined

 Data Guides and Related Resources

Program Name	Purpose
Data Catalog	Catalog of dashboards, data sets, and analytics projects
Data for Product Managers	Information to help Product Managers
Data for Product Analysis	Information to help Product Analysts
Analytics Instrumentation Group	Information covering the Analytics Instrumentation team
Data for Marketing Analysts	Information to help Marketing Analysts
Data for Sales Analysts	Information to help Sales Analysts
Data Triage	Daily process to ensure the data platform remains available for analytics.

SECTION: enterprise-data/organization/programs/data-for-product-managers/index.md

This page is intended to help Product Managers at GitLab understand what data is available to them and how they can use it to understand how their product is used. This page primarily covers two topics: how to consume data, and what data is available.

How to Consume Data at GitLab

The user-facing end of GitLab's data stack consists of our BI Tool, Tableau, which is connected to our Snowflake data warehouse. The Tableau handbook page of the Data team handbook has general information about Tableau aimed for a wider GitLab audience.

Useful links for Product Managers

Here are some useful links that we recommend for you to bookmark:

- Data Catalog: Helpful guides on how to use our data
- DBT documentation: Data model documentation
- Service Ping Metrics Dictionary: Service Ping metric definitions and metadata
- Service Ping documentation
- Snowplow Event Dictionary: Snowplow event definitions and metadata
- Product Data Insights handbook: Information about the Product Data Insights team
- Tableau documentation: Information and guides on how to use Tableau
- Analytics Instrumentation Quick Links
- Quick Start Guide for Internal Event Tracking: Comprehensive instructions on how to instrument event tracking and context around GitLab's internal tracking system.
- Usage Data Instrumentation Issue Template: Issue template for product managers or engineering teams looking to track usage of their features.

Getting Tableau Access

In order to gain access to Tableau, you will need to follow the instructions here and open an access request.

- To create your own charts and dashboards, you need to have a Creator or Explorer license. You can read more about the Tableau license types here.

Published Data Sources

Published data Tableau sources are great ways to allow Tableau users to build charts without writing any SQL or modeling. The Data team has created several Published Data Sources in Tableau that have the official "Certified" badge.

For example, Mart Ping Instance can be used to look at Service Ping ping-level details.

How do I know what tables are available?

The Data team uses a tool called dbt for our data transformation layer. A nice feature of dbt is dbt docs, which automatically creates documentation for all of the models in our schema. Our dbt docs instance can be found here.

- Tableau will show a list of all tables available for querying when you form a connection to Snowflake in the Data Source pane.
-
- Each data source has a distinct naming convention for tables (see more about Key Data Sources below).
 - Service Ping models contain pinginstance in the name (ex: dimpinginstance, martpinginstancemetricmonthly)
 - GitLab.com Postgres db event models start with fctevent or martevent (ex: fcteventuserdaily, marteventnamespacemonthly)
 - GitLab.com Postgres db table replicas are named after the source table/content (ex: dimmergerequest, dimnamespace)
 - Snowplow models contain behavior in the name (ex: martbehaviorstructuredevent, fctbehaviorwebsitepageview)

How can I update or add more information to the dbt docs?

You will need to locate the file you wish to update or create in the gitlab-data analytics project. Please be sure to read and follow the SQL style guide when creating the changes. If you wish to update only the descriptions or information about tables you will be looking for a schema.yml file. If you wish to actually change the structure of tables it will be a .sql file.

Next, create a branch and then submit an MR using the dbt Model Changes template to the gitlab-data analytics project. When creating your branch and MR please follow the Data team workflow and use the appropriate Data team labels.

How can I get help?

If you ever get stuck or have a question, please ask for help in the data slack channel. It is recommended to also cross-post questions in your g, s, or product channels because many PMs have data related expertise and can provide you quick assistance for common product data questions.

- Remember, it's helpful for us to know the context behind your question. Don't just say what you want to know but also why so others can point you to a more efficient way to get your answer.
- This document is meant to serve as a guide of best practices. Please add what you learn when you need help with this content.

If needed, you may create an issue in the Product Data Insights project and assign it to a product data analyst. You can read more about working with the PDI team [here](#).

Guidance for Instrumenting Feature Tracking

As a GitLab PM, you're responsible for defining and tracking metrics for your team's features. This guide will walk you through the process, tools, and resources available to help you succeed.

Quick links for Instrumenting Feature Tracking

- CLI generator to automatically create event and metric definition files: An interactive CLI that gathers your requirements, automatically generates event and metric definition files, and produces ready-to-use instrumentation code for engineers to implement and test
- Quick Start Guide for Internal Event Tracking: Comprehensive instructions on how to instrument event tracking and context around GitLab's internal tracking system
- Getting Started Standard Context Fields: Documentation on each standard context field included in Internal Event Tracking and descriptions of their intent
- Usage Data Instrumentation Issue Template: Issue template for product managers or engineering teams looking to track usage of their features
- Product Data Insights Performance Indicator Chart Issue Template
- Product Data Insights Ad Hoc Analysis Issue Template

Self-Service Feature Tracking Dashboards

If your analytics needs for your new or recently modified feature are met by these dashboards,

you can skip creating a Product Data Insights (PDI) Issue:

- PD: Centralized Product Usage Metrics
- PD: Product Usage Metrics (.com & Service Ping)
- PD: Firmographic Product Metric Usage
- PD: Subscription Feature Usage Trends
- AI Gateway Reporting

Process for Instrumenting Feature Tracking

1. Plan Your Analytics Requirements

Owner: Product Manager

- Start by determining what you need to measure:
 - What user behaviors indicate feature success?
 - What metrics will help you make product decisions?
 - What data points do you need for your team's KPIs?
- If existing dashboards don't meet all of your needs, create a Product Data Insights (PDI) Issue to request additional analytics. Product Data Insights (PDI) Issue

1. Create Instrumentation Issue

Owner: Product Manager

Option A: Use the CLI Generator to generate requirements for your Instrumentation Issue

- Use the CLI generator tool
- Benefits:
 - Automatically generates event and metric definition files
 - Produces ready-to-use instrumentation code
 - Reduces implementation time for engineers
 - Ensures consistency with GitLab's tracking standards

Option B: Use Usage Data Instrumentation Issue Template to outline metric requirements

- Use the Usage Data Instrumentation Issue Template

Tag your assigned product analyst to review metric properties

1. Implement Tracking

Owner: Engineer

- Create an Internal Events Tracking Merge Request (MR)
- Implement new metrics according to specifications defined in Usage Data Instrumentation Issue

1. Test and Validate

Owner: Engineer

- Perform local testing
- Request review from Analytics Instrumentation team member
- Verify test events match properties defined in the Issue

1. Create Analysis

Owner: Product Analyst

- For analyses requiring user and event grain GitLab.com data (Snowplow), data collection will be sufficient for analysis 1-2 weeks after MR merge
- For analyses requiring aggregated SM and Dedicated data (Service Ping), data collection will be sufficient for analysis 6-8 weeks after MR merge due to minimum version adoption requirement for Service Ping metrics
- Complete requirements specified in PDI Issue (if applicable)

Special Considerations for AI Gateway Features

When instrumenting features routed through the AI Gateway, follow these guidelines:

1. Represent new features routed through the AI Gateway as unit primitives

- New distinct features should be represented as a unit primitive
- To add a new unit primitive, reach out to ~"group::cloud connector"

1. Set up tracking for the new unit primitive

- Contact ~"group::analytics instrumentation" with the necessary fields to be tracked
- For more information on AI Gateway instrumentation, see the documentation
- Once instrumented, AI Gateway events using the unit primitive framework:
 - Cannot be blocked by users
 - Are tracked at the event grain for all deployment types

1. For more granular reporting

- If you need more detail than a 'request' of the AI Gateway at the broad feature grain, use Internal Events Tracking

- Internal events can be connected to unit primitive events using a correlationid for behavior funnel or more granular reporting use cases (GitLab.com only)

1. Viewing AI Gateway data

- AI Gateway Reporting automatically displays new unit primitive requests once they have been instrumented by ~"group::analytics instrumentation"
- Additional analytics can be requested by creating a Product Data Insights (PDI) Issue

Key Contacts and Resources

- For questions about the feature tracking process, reach out to gmonitoranalyticsinstrumentation.
- The Analytics Instrumentation team owns our internal product feature tracking system.

By following this process and understanding the roles involved, PMs can effectively instrument and track metrics for their features, enabling data-driven decision-making and product improvement.

Key Data Sources for Product Managers at GitLab

We have three primary data sources for product usage data:

- Service Ping (for Self-Managed, Dedicated, and GitLab.com)
- GitLab.com Postgres Database (for GitLab.com)
- Snowplow (for GitLab.com and the AI Gateway)

Each data source comes with its own caveats, capabilities, and limitations. The first question we on the Data or PDI teams ask product managers is usually "are you interested in knowing this for Self-Managed or GitLab.com?" Our approach to answering your question and the data source(s) available differ greatly between the two. Although our Self-Managed offering has many more active customers, our GitLab.com offering has much more granular data available to analyze.

Service Ping (Version App)

Service Ping is a custom tool that GitLab built to collect weekly aggregated information from our customers across various deployment options:

- Self-Managed: Customers who host our product on their own hardware.
- GitLab Dedicated: Our fully-managed, single-tenant SaaS offering where each customer gets their own isolated instance of GitLab, hosted and managed by GitLab's team.
- GitLab.com: Our multi-tenant SaaS offering.

Key Concepts

- Service Ping collects and reports data at an installation level. For Self-Managed and GitLab Dedicated customers, this means one ping per installation per week. For GitLab.com, which is a single, large-scale installation, Service Ping reporting still occurs as a single ping within our Service Ping framework, representing the entire GitLab.com ecosystem. (In other words, despite the deployment type or installation size, we still receive data at the installation level).
- Service Ping provides pre-aggregated counts of specific events/actions (aka metrics). We are unable to do any analysis at a more granular level (ex: user-level, project-level, etc) since the metrics are already aggregated.
- Sending Service Ping is optional but defaults to being on.
 - You can see the percent of paid subscriptions that successfully send a ping every month on this chart. We do not know about the opt-in rate of unlicensed (Core/Free) users but assume the same rate. Here is more information on why some installations block data from being sent.
 - Since we do not receive pings from every customer, we apply an estimation in monthly reporting to fill in the gaps. You can read more about the estimation methodology in the internal handbook [here](#).
- Customers must adopt a version of GitLab with a metric instrumented in order to report the metric. For example, if a metric is added in 17.3, only customers on versions ≥ 17.3 will report the metric. This means that it can take months to have a sufficient number of customers reporting the metric.
 - You can track GitLab version adoption on this [dashboard](#).
- Pings are added to Snowflake daily. By the 2nd of the month all the data should be available from the previous month.

Key Columns

- pingdeploymenttype is the best column to use in order to differentiate Self-Managed, Dedicated, and GitLab.com usage.

- If using pingdeliverytype, know that both Dedicated and GitLab.com are included in SaaS.
- Use the metricspath column to filter to the metric of interest. You can find more metric-level details and metadata in the Service Ping Metrics Dictionary.
- For monthly reporting, we limit to the last ping we receive each month per installation. You can filter your queries to islastpingofmonth = TRUE
- We often talk about "installations" or refer to the diminstallationid column. An "installation" is the unique combination of diminstanceid/uuid and dimhostid.
- We use "installations" since a single "instance" (diminstanceid/uuid) can have multiple hosts.

Examples

Here is an example of a query that provides ping-level details, filters out GitLab.com, and limits to the last ping of the month:

```
{{% details summary="Self-Managed and Dedicated Ping-Level Details"%}}
```

```
sql
SELECT
FROM commonmart.martpinginstance
WHERE pingcreatedat >= CURRENTDATE-30
  AND pingdeploymenttype != 'GitLab.com'
  AND islastpingofmonth = TRUE
LIMIT 1000
;
```

```
{{% /details %}}
```

Here is an example of a query that provides metric-level reporting by month and deployment type:

```
{{% details summary="Service Ping Metric by Month and Deployment Type"%}}
```

```
sql
SELECT
  pingcreateddatemonth,
  pingdeploymenttype,
  metricspath,
  SUM(monthlymetricvalue) AS monthlymetricvalue,
  COUNT(DISTINCT IFF(monthlymetricvalue > 0, diminstallationid, NULL)) AS installationcount
--count of installations reporting usage that month
FROM commonmart.martpinginstancemetricmonthly --this model already filters to the last ping of
the month
WHERE pingcreateddatemonth >= '2024-06-01'
  AND metricspath = 'usageactivitybystagemonthly.secure.sastscans' --arbitrary metric, switch
this out for metric of interest
GROUP BY 1,2,3
ORDER BY 1,2
;
```

{{% /details %}}

GitLab.com Postgres Database

Because GitLab.com is a GitLab instance hosted by GitLab, we have access to the instance's postgres database and can load parts of it into our Snowflake data warehouse. This means we can get a very detailed look into how our product is used on GitLab.com.

- Any part of the product that creates a table on the backend (see the schema file) can be added to the ELT job which will sync 3x a day to the warehouse. From there, all we need to do is build a dbt base model to make them accessible in Tableau.

What if the table or column I want isn't in the data warehouse?

Our ELT process works by explicitly stating which columns and tables we want to import into the data warehouse. This means we might be missing a column or whole table that you want to have in the data warehouse for analysis. When this is the case, please create a Data issue letting us know what you want us to import using the New Data Source template. Before doing so, please confirm that the table/column is truly part of the production schema.

Replicating Service Ping using GitLab.com Data

- Because Service Ping only aggregates data at an installation level, it is not super useful for GitLab.com since we often want to see information at the namespace level. For example, knowing that 40K people used your stage on GitLab.com is somewhat useful, but you'll probably want to know more context (Are they free or paid? What plan are they on? Do I have any power users or is usage equally distributed?)
- But since we have access to the GitLab.com Postgres database, we are capable of replicating many Service Ping metrics at the namespace level or even the user level.
- This model exemplifies how some Service Ping metrics could be replicated for GitLab.com at the namespace level. This model is available as a Published Data Source in Tableau as Mart Event Valid.

Examples

Here is an example of a query that will generate GitLab.com UMAU by day:

{{% details summary="GitLab.com UMAU by Day"%}}

```
sql
SELECT
  eventdate,
  COUNT(DISTINCT dimuserid) AS umau
FROM commonmart.marteventuserdaily
WHERE eventdate >= CURRENTDATE-30
  AND isumau = TRUE
GROUP BY 1
ORDER BY 1
;
```

{{% /details %}}

Here is an example of a query that will generate paid GitLab.com GMAU by month:

{{% details summary="Paid GitLab.com GMAU by Month"%}}

```
sql
SELECT
  eventcalendarmonth,
  groupname,
  usercount
FROM commonmartproduct.rpteventxmaumetricmonthly
WHERE eventcalendarmonth >= '2024-06-01'
  AND isgmau = TRUE
  AND usergroup = 'paid'
ORDER BY 2,1
;
```

{{% /details %}}

Snowplow

Snowplow Analytics is an open-source enterprise event-level analytics platform that enables data collection from multiple platforms for advanced data analysis.

- We pseudonymize userid on all Snowplow events, meaning that we are unable to connect an event to a specific user (or the GitLab.com Postgres db).
- We also pseudonymize page URLs to remove any potential PII or RED data.
- Self-Managed and Dedicated installations do not send Snowplow data to GitLab.
- The one exception is the AI Gateway where we receive events from all deployment types (Self-Managed, Dedicated, and GitLab.com).

Key Concepts

- Because Snowplow does not rely on Service Ping, we do not need to wait for a version of GitLab to be adopted to start receiving data. We can collect and visualize data as soon as the instrumentation is deployed.
- Even though the pseudonymization of userid of Snowplow events is a limitation, with the fast feedback, Snowplow is an effective source of data to measure feature adoption and usage.
- Note: We are still able to count the number of users who engage with a feature, which is sufficient for most use cases. We just do not know who those users are.
- Snowplow events can be blocked by the user.

Instrumentation

Analytics Instrumentation has built Internal Event tracking, which will guide you on how to instrument Snowplow events. To get started, use the Quick Start Guide to Internal Event Tracking.

Once your Snowplow events have been instrumented, as part of the validation process, the newly instrumented event should be tested to ensure it is working properly. While you as the PM probably won't be doing the validation yourself every time, it is nice to know how it works. You can learn more about testing Snowplow events in the Internal Event documentation [here](#).

Visualize your events in Tableau

The data you have instrumented is most useful if it can be visualized in a chart. Refer to the Tableau section of the handbook for information on creating charts.

- First, check if they are correctly stored in Snowflake in the Snowplow Event Exploration Dashboard (note: the data volume is quite large, so please be patient with dashboard load time). You can use the filters to find your events. If you are not sure of the value of the different attributes, they should have been captured in the event definition. If not, check with your engineering manager.
- Once you have verified that your events are properly stored, you are ready to query and visualize the data! Please be aware that we are collecting several millions of events (page views, structured events) per day, so the whole dataset is quite slow to query. In order to make it easier to explore this data source, we have created several smaller tables:
 - commonmart.martbehaviorstructuredevent: contains ALL structured events (these are what is instrumented with Internal Event tracking)
 - common.fctbehaviorwebsitepageview: contains ALL page views
 - common.fctbehaviorunstructuredevent: contains ALL unstructured events (including click events, form submissions, etc)

Examples

```
{{% panel header="PRO TIP: Optimizing queries" header-bg="info" %}}
```

The Snowplow models are quite large and can be slow to query. To make your query faster, use a date (behaviorat) filter in your WHERE statement.

```
{{% /panel %}}
```

Here is an example of a query that will look at the top 100 Snowplow events from the last several days:

```
{{% details summary="Top 100 Snowplow Events"%}}
```

```
sql
SELECT
  eventaction,
  COUNT() AS eventcount
FROM commonmart.martbehaviorstructuredevent
WHERE behaviorat >= CURRENTDATE-3
GROUP BY 1
ORDER BY 2 DESC
LIMIT 100
;
```

{{% /details %}}

Here is an example of a query that will look at top 100 pages viewed in the last several days (note how the URLs are pseudonymized):

{{% details summary="Top 100 Pages Viewed"%}}

```
sql
SELECT
  pageurl,
  COUNT() AS pageviewcount
FROM common.fctbehaviorwebsitepageview
WHERE behaviorat >= CURRENTDATE-3
GROUP BY 1
ORDER BY 2 DESC
LIMIT 100
;
```

{{% /details %}}

Other data sources\\

- Zuora
 - Zuora is our SSOT for subscription information and key metrics like ARR and Seat Count.
- Customers Dot (CDot) database
 - CDot is our SSOT for Fulfillment-related data, including trials.
- Sheetload
 - You can load your own Google Sheets into the data warehouse. [Read more here.](#)

Analytics Instrumentation

Analytics Instrumentation is part of the product org and is completely separate from the Data and Product Data Insights teams. However, these teams collaborate closely as the Customer Product Adoption pod.

- The Analytics Instrumentation team members are the DRIs for data collection across both GitLab.com and Self-Managed. They own Service Ping and Snowplow. They are the ones to go to for questions like:
 - How do I instrument a new metric for Self-Managed?
 - What are the best practices for adding to Service Ping?
 - How can I use Snowplow/Internal Events to track a frontend interaction on GitLab.com?
 - Can I utilize Snowplow/Internal Events to track events on the server-side?

SECTION: enterprise-data/platform/_index.md

Our Data Platform Vision

These ambitions are set to be a guiding vision for GitLab's data platform.

Makes it Easier to Contribute

Contributing to GitLab's Data Platform is easy and using the platform is intuitive

Documentation is complete and relevant for users and contributors

All data transformations are implemented in dbt

CI/CD is seamless, intuitive, automated for contributors and reviewers

Data state is derivative from sources and transformations

Data pipelines are idempotent

Is Reliable

The data platform along with the data it delivers is consistent in availability and accuracy

All breaking changes are testable in Dev and/or Staging Environments

Automated tests are implemented at every stage of the data delivery process

Every component of the platform can and should be defined in code and version controlled

Is Secure

The Data Platform doesn't put people at risk

Data is only accessible to those authorized by documented approvals

The GitLab data team will stick with the Principle of Least Privilege regarding the authorization and authentication

Is Maintainable

The Data Platform component will be created with considering the good engineering practices for the ease of maintainability. It means tracking of maintainability is intended to help reduce or reverse a system's tendency toward "code entropy" or degraded integrity

Benefits a Larger Community

GitLab's Data Platform is relevant to a community larger than GitLab's and depends on a larger community of engineers.

Relevant Platform code is open sourced

Platform enhancements are contributed back into community projects

We prefer generalizable specifications and standards over idiosyncratic custom development

Purpose

The Data Platform is used for data analytics purposes. This document conceptually describes on high level the components which all together are defined as the Data Platform.

Scope

This document is limited to describe the Data Platform conceptually. There are other resources that describe it in more detail (i.e. the data pipelines and the infrastructure).

Roles & Responsibilities

Role	Responsibility
GitLab Team Members	Responsible for taking notice of how the standard that forms the Data Platform
Data Platform Team Members	Responsible for implementing and executing data use cases based on this standard
Data Management Team	Responsible for approving significant changes and exceptions to this standard

Standards

Quick Links

Data Infrastructure
Data Pipelines
Data CI Jobs
dbt Guide
Enterprise Data Warehouse
Data Pump
Jupyter Guide
Permifrost
Python Guide
Python/Tools package management and inventory
Snowflake
Snowplow
SQL Style Guide
Meltano
R/RStudio
Tableau

Our Data Stack

We use GitLab to operate and manage the analytics function.

Everything starts with an issue.

Changes are implemented via merge requests, including changes to our pipelines, extraction, loading, transformations, and parts of our analytics.

Stage	Tools
Extraction	Stitch, Fivetran, Tableau Prep, and Custom Code
Loading	Stitch, Fivetran, Tableau Prep, and Custom Code
Orchestration	Airflow, Tableau Prep
Data Warehouse	Snowflake Enterprise Edition

Transformations	dbt and Python scripts
Data Visualization	Tableau
Advanced Analytics	jupyter

fas fa-exchange-alt fa-fw -text-purple Extract and Load

We currently use Stitch and Fivetran for some of our data sources. These are off-the-shelf ELT tools that remove the responsibility of building, maintaining, or orchestrating the movement of data from some data sources into our Snowflake data warehouse. We run a full-refresh of all of our Stitch/Fivetran data sources at the same time that we rotate our security credentials (approx every 90 days). Prior to running a full refresh we will drop all of the tables.

Stitch and Fivetran handle the start of the data pipeline themselves. This means that Airflow does not play a role in the orchestration of the Stitch- and Fivetran schedules.

Other solutions we use to extract data are:

1. Meltano
1. Custom pipelines built in Python and orchestrated via Airflow
1. Flows built in Tableau Prep and orchestrated by Tableau Cloud
1. Snowflake data share

For source ownership please see the Tech Stack Applications data file.

Data Sources

The following table indexes all of the RAW data sources we are loading into the data warehouse from external locations. We manage the development backlog and priorities in the New Data Source/Pipeline Project Management sheet, with links to GitLab issues for up-to-date status and progress management. The new data source handbook page describes how the Data Team handles any request for new data sources.

The link in the pipeline column in the table below will bring you to the detailed page of the specific data pipeline if applicable.

Key

Pipeline: The technology we use to replicate data.

RF (Replication Frequency): How often we load new and updated data.

Raw Schema: The schema in the RAW database where data is stored.

Prep Schema: The schema in the PREP database where source models are materialized.

Audience: The primary users of the data.

SLO: Service Level Objective. Our SLO is the time between real-time and the data made available for consumption.

Technically, this means the time between when an entry is made in an upstream system and when the data is available in the Snowflake PROD layer (which includes transformations in dbt). x indicates undefined or not run

{{% all-data-warehouse-sources %}}

<!-- Add or edit data sources in <https://gitlab.com/gitlab-com/www-gitlab->

com/-/tree/master/data -->

Source contacts

See the source contact spreadsheet for who to contact if there are any external errors.

Tier definition

| Aspect | Tier 1 | Tier 2 | Tier 3 |

|:-|:-|:-|:-|

| Description | - Trusted Data solutions that are most important and business critical.

 - Components needs to be available and refreshed to ensure day-by-day operation | -

Data solutions that are important and beneficial for gathering insights.

 - Components should be available and refreshed to supporting day-by-day operation | - Data solutions that are important for Ad-Hoc, periodically or one-time analysis.

 - Components could be unavailable or data not refreshed. |

|Criteria|- Any data, process, or related service that would result in a \$100k or higher

business impact if unavailable for 24 hours

- Affecting more than 15 business users |

- Any data, process, or related service that would result in less than \$100k business impact if unavailable for 24 hours

- Affecting between 5 and 15 business users | - Any data, process or related service that would not result in a immediate business impact if unavailable for more than 5 working days

- Affecting less then 5 business users |

|Impact due to outage|Severe|Lenient|Negligible |

Data Team Access to Data Sources

In order to integrate new data sources into the data warehouse, specific members of the Data team will need admin-level access to data sources, both in the UI and through the API.

We need this admin-level access through the API in order to pull all the data needed to build the appropriately analyses and through the UI to compare the results of prepared analyses to the UI.

Sensitive data sources can be limited to no less than 1 data engineer and 1 data analyst having access to build the require reporting.

In some cases, it may only be 2 data engineers.

We will likely request an additional account for the automated extraction process.

Sensitive data is locked down through the security paradigms listed below;

Data Source Overviews

Customer Success Dashboards

Netsuite

Netsuite and Campaign Data

Version (pings)

Note that up until October 2019, the data team referred to the entire version data source as "pings". However, usage ping is only one subset of the version data source which is why we now use "version" or "version app" to refer to the version.gitlab.com data source and "usage data" or "usage pings" or "pings" to refer to the specific usage data feature of the version data source. In the context of Data extraction, when it comes to Service ping data ingestion,

specific details should be found in the Service ping page or in the Readme.md page for Service ping

Salesforce
Zendesk

Snowplow Infrastructure

Refer to the Snowplow Infrastructure page for more information on our setup.

*<i class="fas fa-clock fa-fw -text-orange"></i>*Orchestration

We use Airflow on Kubernetes for our orchestration. Our specific setup/implementation can be found here. Also see the Data Infrastructure page for more information.

*<i class="fas fa-database fa-fw -text-purple"></i>*Data Warehouse

We currently use Snowflake as our data warehouse. The Enterprise Data Warehouse (EDW) is the single source of truth for GitLab's corporate data, performance analytics, and enterprise-wide data such as Key Performance Indicators. The EDW supports GitLab's data-driven initiatives by providing all teams a common platform and framework for reporting, dashboarding, and analytics. With the exception of point-to-point application integrations all current and future data projects will be driven from the EDW. As a recipient of data from a variety of GitLab source systems, the EDW will also help inform and drive Data Quality best-practices, measures, and remediation to help ensure all decisions are made using the best data possible.

Snowplow updating columns

Snowplow nullify geo columns

Issue: Snowflake documentation

In order not to extract geo data into Snowplow, the following columns were nullified:

geozipcode
geolatitude
geolongitude
useripaddress

This nullified is applied in Snowplow from 2023-02-01 and the files have the same structure, just column values are set to NULL. The Data Team updated old files and set mentioned columns to NULL, and also set columns to NULL in Snowflake. This is applicable to the RAW, PREP and PROD layers in Snowflake.

As desired to avoid a duplicate load of the updated files in the S3 bucket as per Snowflake documentation, the folder structure is modified from:

```
bash
- gitlab-com-snowplow-events/
  output/ <---- all files are located here
    2019/01/01
```

...
(present day)

to the new structure:

```
bash
- gitlab-com-snowplow-events/
  outputnullifiedcolumns/ <---- all files are nullified and updated
    2019/01/01
    ...
    2023/01/31
  output/ <---- new files will land here and will be loaded by Snowpipe
    2023/02/01
    ...
    (present day)
```

Snowplow nullify pageurlpath columns

Issue: s3: Pseudonymize pageurlpath in Snowflake and s3 bucket

In order to be compliant with data into Snowplow, the following columns were pseudo-anonymized:

pageurlpath

This pseudo-anonymization is applied for Snowplow data, for the period 2022-10-26 - 2024-12-01 and the files have the same structure, just column values are pseudonymized.

The Data Team updated old files and pseudo-anonymized pageurlpath column, and also pseudo-anonymized pageurlpath column in Snowflake.

This is applicable to the RAW, PREP and PROD layers in Snowflake.

As desired to avoid a duplicate load of the updated files in the S3 bucket as per s3:
Pseudonymize pageurlpath in Snowflake and s3 bucket, the folder structure is modified from:

```
bash
- gitlab-com-snowplow-events/
  outputnullifiedcolumns/ <---- all files are nullified and updated (in the previous
iteration)
    2022/10/26
    ...
    2023/
      02/
output/
  2023/
    02/
    03/
```

to the new structure:

```
bash
- gitlab-com-snowplow-events/
  outputnullifiedcolumns/
    2019/01/01
    ...
    2022/10/25
  outputmaskpageurlpath/ <---- all files are pseudonimized
    2022/10/26
    ...
    2023/12/01
  output/ <---- new files will land here and will be loaded by Snowpipe
    2023/12/02
    ...
    (present day)
```

> Note: All new loads in the S3 bucket will go into the same folder as before gitlab-com-snowplow-events/output.

Snowflake support portal access

To get access to snowflake support portal, please follow the below steps.

When you are in your Snowsight instance, open your account (bottom-left corner) and go to the Support option

On the panel, you can see the already open cases

In the top-right corner, to open a new case, press + Support Case button
Fill in the data to describe your issue and the Snowflake team will handle it

For each update on your case, you will be informed by email

Warehouse Access

To gain access to Snowflake:

Create an issue in the access requests project documenting the level of access required.
Do not request a shared account - each account must be tied to a user.

We loosely follow the paradigm explained in this blog post around permissioning users.
When asking to mirror an existing account, please note that access to restricted SAFE data will not be provisioned/mirrored (currently provided via restrictedsafe role).

Snowflake is part of the Access Review Procedure and manager will be asked on a quarterly basis to review the access their team members have in Snowflake. It is expected from the

manager to understand the available roles(structure) in Snowflake if approving an AR or reviewing their team member access.

In the access review, only the first level of Snowflake roles are reported (the ones that are directly attached to the user). I.e. If a team member does have the analystmarketing role, only the analystmarketing is reported and all inherited roles in the analystmarketing are not.

Roles could be distinguished between functional roles and object roles

See this list of functional roles in Snowflake and object roles.

Object roles are directly related to systems and gives Team Members access to all of the data we extract from those upstream source systems.

To know in all detail what a role entails check this YAML file.

If unsure, during AR process or Access Review, please reach out to a Data Platform Team Member to understand in detail what a specific role entails.

Snowflake Analyst

Snowflake can be used to perform analyses on the data that is available by writing SQL-code. Anything created and any outcome of the analyses is considered as an ad-hoc analyses. It is important to know that anything that is created (i.e. worksheets and dashboards) is not version controlled and not supported or managed by the Central Data Team. I.e. When a team member off-boards from GitLab, the worksheets and dashboards are not accessible anymore. In order to persist analyses, team members can build Tableau workbooks, store code snippets in a GitLab project, or commit code to the Data Team's dbt project.

In order to be granted access to Snowflake, an AR must be opened as described. A new user will be created with access to query the PROD database.

There are 2 levels of data access:

General data --> Adding the Snowflake snowflakeanalyst role to their account.

SAFE data (you must be or will become a designated insider) --> Adding the Snowflake snowflakeanalystsafe to their account. See the SAFE Guide for the needed approvals.

All users will have access to devxs and reporting -(size M) warehouse. When creating the user, the devxs warehouse as default warehouse.

Snowflake Permissions Paradigm

We use Permifrost to help manage permissions for Snowflake.

Our configuration file for our Snowflake instance is stored in this roles.yml file.

Also available is our handbook page on Permifrost.

We follow this general strategy for role management:

Every user has an associated user role

Functional roles exist to represent common privilege sets (analystfinance, datamanager, productmanager)

Logical groups of data have their own object roles

Object roles are assigned primarily to functional roles

Higher privilege roles (accountadmin, securityadmin, useradmin, sysadmin) are assigned directly to users

Service accounts have an identically named role

Additional roles can be assigned either to the service account role or the service account itself, depending on usage and needs

Individual privileges can be granted at the granularity of the table & view

Warehouse usage can be granted to any role as needed, but granting to functional roles is recommended

User Roles

Every user will have their own user role that should match their user name.

Object level permissions (database, schemas, tables) in Snowflake can only be granted to roles.

Roles can be granted to users or to other roles.

We strive to have all privileges flow through the user role so that a user only has to use one role to interact with the database.

Exceptions are privileged roles such as accountadmin, securityadmin, useradmin, and sysadmin.

These roles grant higher access and should be intentionally selected when using.

Functional Roles

Functional roles represent a group of privileges and role grants that typically map to a job family.

The major exception is the analyst roles.

There are several variants of the analyst role which map to different areas of the organization.

These include analystcore, analystfinance, analystpeople, and more.

Analysts are assigned to relevant roles and are explicitly granted access to the schemas they need.

Functional roles can be created at any time.

It makes the most sense when there are multiple people who have very similar job families and permissions.

Functional Role Assignment

This list of functional roles gives a high level understanding of what the role entails. If missing or to know in all detail what a role entails check this YAML file.

Functional Role	Description	SAFE Data Y/N
---	---	---
datateamanalyst	Access to all PROD data, sensitive marketing data, Data Platform metadata and some sources.	Yes
analystcore	Access to all PROD data and meta data in the Data Platform	No
analystengineering	Access to all PROD data, meta data in the Data Platform and Engineering related data sources.	Yes
analystgrowth	Access to all PROD data, meta data in the Data Platform and various data sources.	Yes
analystfinance	Access to all PROD data, meta data in the Data Platform and finance related data sources.	Yes
analystmarketing	Access to all PROD data, meta data in the Data Platform and marketing related data sources.	Yes
analystpeople	Access to all PROD data, meta data in the Data Platform and various related	

data sources, including sensitive people data. | Yes |
analystsales	Access to all PROD data, meta data in the Data Platform and various related data sources	Yes
analystsupport	Access to PROD data, meta data in the Data Platform and raw / prep Zendesk data, including sensitive Zendesk data	No
analyticsengineercore	A combination of analystcore, datateamanalyst role with some additions	Yes
datamanager	Extension access to Snowflake data	Yes
engineer	Extension access to Snowflake data to perform data operation tasks in Snowflake	Yes
snowflakeanalyst	Access to PROD data in Snowflake, EDM schema and workspaces	No
snowflakeanalystsafe	Access to PROD data in Snowflake, EDM schema and workspaces including SAFE data	Yes
sensitivepiidataviewer	Access to all sensitive fields in person and contact data mastery models.	No

Object Roles

Object roles are for managing access to a set of data.

Typically these represent all of the data for a given source.

The zuora object role is an example.

This role grants access to the raw Zuora data coming from Stitch, and also to the source models in the prep.zuora schema.

When a user needs access to Zuora data, granting the zuora role to that user's user role is the easiest solution.

If for some reason access to the object role doesn't make sense, individual privileges can be granted at the granularity of a table.

Masking Roles

Masking Roles manage how users interact with masked data. Masking is applied at the column level and which columns are masked is the decision of the source system owner. Masking is applied to a column in a schema.yml file within the dbt code base when a data object is created via dbt. As some users will need access to unmasked data the masking role allows for permissions to the unmasked data to be granted on a functional or object role level. For example if the masking role of peopledatamasking is applied to the column locality then the functional role of analystpeople can be set as a member of the peopledatamasking role to allow the analysts to see unmasked people data.

When a masking policy is created, it is created based on the masking roles and only one masking policy can be applied to a column.

Examples

This is an example role hierarchy for an Data Analyst, Core:

mermaid

graph LR

A([User: datwood]) -->|Member of| B[User Role: datwood]

B -->|Member of| C[Functional Role: analystcore]

```

C -->|Member of| D[Object Role: workday]
C -->|Member of| H[Object Role: dbtanalytics]
C -->|Member of| E[Object Role: netsuite]
C -->|Member of| F[Object Role: zuora]
G{{Privileges: analyticssensitive}} -->|Granted to| C

```

This is an example role hierarchy for an Data Engineer and Account Administrator:

```

mermaid
graph LR
  A([User: tmurphy]) -->|Member of| B[User Role: tmurphy]
  B -->|Member of| C[Functional Role: engineer]
  C -->|Member of| F[Functional Role: loader]
  C -->|Member of| H[Functional Role: transformer]
  G{{ Privileges: Read/Write Raw}} -->|Granted to| C
  A -->|Member of| D[Privileged Role: sysadmin]
  A -->|Member of| E[Privileged Role: securityadmin]

```

This is an example role hierarchy for a Security Operations Engineer:

```

mermaid
graph LR
  A([User: ssichak]) -->|Member of| B[User Role: ssichak]
  A -->|Member of| C[Privileged Role: securityadmin]

```

Managing Roles for Snowflake

Here are the proper steps for provisioning a new user and user role:

Make sure we have an issue in the GitLab Data Team project linking the original request with the Provisioning label applied

Login to Snowflake and switch to securityadmin role

All roles should be under securityadmin ownership

Copy the userprovision.sql script and replace the email, firstname, and lastname values in the initial block

If a password is needed, use Password Generator to create one

Send username and password credentials to user with One Time Secret or via Slack

Document in Snowflake roles.yml permifrost config file (this file is automatically loaded every day at 12:00a.m. UTC)

Add the user and user role you created

Assign the user role to new user

Assign any additional roles to user

Ensure the user is assigned the application in Okta

Ensure the user is assigned to the okta-snowflake-users Google Group

Here are the proper steps for deprovisioning existing user:

Snowflake deprovision should be done via an offboarding issue or access request issue. Make sure we have an issue in the GitLab Data Team project linking the original source request with the Deprovisioning label applied.

Login to Snowflake and switch to securityadmin role

All roles should be under securityadmin ownership.

Copy the userdeprovision.sql script and replace the USERNAME. The reason for not removing and leaving the user in snowflake and setting disabled = TRUE is to have a record of when the user lost access.

Remove the user from okta-snowflake-users Google Group

Remove the user records in Snowflake roles.yml permifrost config file (this file is automatically loaded every day at 12:00a.m. UTC)

For more information, watch this recorded pairing session (must be viewed as GitLab Unfiltered).

Snowflake Provisioning Automation

In FY25-Q1, we are moving towards semi-automating the above Managing Roles for Snowflake process, OKR epic. This will enable all GitLab Team Members to create a Snowflake user themselves with minimal support by the Data Platform Team. This will speed up the provisioning process and shorten the time a GitLab Team member can get access to Snowflake.

All GitLab Team Members are encouraged to open a MR following this runbook if they need access to Snowflake.

High-level description of the process:

1. Open an Access Request and get the approvals in place
1. Open an MR
1. Run CI pipeline
1. Review from Data Platform Team codeowner.

The rest of the section is meant to describe the automated process in more detail.

The main processes that have been automated are:

1. create/remove users from Snowflake platform
1. update roles.yml which is used by Permifrost to update Snowflake role/user permissions

Both of these processes will be made accessible via CI jobs so that the user can potentially self-serve, requiring just MR review/approval from a data engineer.

Both CI jobs follow a common pattern, the end user simply has to add/remove users from within the snowflakeusernames.yml file, and the CI job will run based on the changes to the file.

- 1) Automate creating users/roles in Snowflake platform

Prior to running Permifrost, the users/roles need to be first created in Snowflake.

The snowflakeprovisioningsnowflakeusers CI job allows the user to create these users/roles in

Snowflake.

See the CI jobs page for more information on the available arguments and default values.

2) Automating roles.yml

Once the users/roles have been created in Snowflake, roles.yml needs to be updated to reflect the desired permissions.

The snowflakeprovisioningrolesyaml CI job allows the end user to automatically update roles.yml with the desired permissions.

See the CI jobs page for more information on the available arguments and default values.

Furthermore, the next section provides additional details on optional templated arguments within snowflakeprovisioningrolesyaml CI job:

<details><summary>Optional Templated Arguments</summary>

Custom Templates

This is useful if you have many users that need a value different from the default. One option would be to run with the default values, and then manually update the MR, but depending on the number of users to update, a potentially better option is to pass in a custom values template.

The rest of the section will do two things:

1. Explain how templates work

1. For convenience, provide custom templates that represent common values currently used in roles.yml

To illustrate how templates work, let's start with an example. This is the default roles template:

```
json
{
  "{{ username }}": {
    "memberof": [
      "snowflakeanalyst"
    ],
    "warehouses": [
      "devxs"
    ]
  }
}
```

This is valid JSON, but note that it is templated. That is, {{ username }} is a Jinja template, and the template will be later rendered to an actual value within the script.

Now, an example of when we want to override the default value above. What happens if for the next batch of users, we want them to also have devm warehouse?

Within the CI job, we could pass in a custom template to override the default value like so:

plain

```
ROLETEMPLATE: {"{{username}}": {"memberof": ["snowflakeanalyst"], "warehouses": ["devxs", "devm"]}}
```

Currently, these are the available template-able values that will be rendered:

```
{{ username }}
{{ proddb }}
{{ prepdb }}
{{ prodschemas }}
{{ prepschemas }}
{{ prodtables }}
{{ preptables }}
```

Common Custom Templates

This section is meant to provide custom templates (non-default values) that represent common-occurring values in roles.yml that can be copy/pasted for use.

Default denotes that this is the template used if not explicitly overridden.

Common denotes that while the template is not used by default, these values are still commonly used within roles.yml

Databases

Default: None, no databases are added

Common: CI job argument to create a personal prep/prod database for each user:

```
sh
DATABASETEMPLATE: [{"{{ proddatabase }}": {"shared": false}}, [{"{{ prepdatabase }}": {"shared": false}}]
```

Roles

Default:

```
sh
ROLETEMPLATE: {"{{ username }}": {"memberof": ["snowflakeanalyst"], "warehouses": ["devxs"]}}
```

Common- CI job argument to create a role for a data engineer:

sh

```
ROLETEMPLATE: {"{{ username }}": {"memberof": ["engineer","restrictedsafe"],"warehouses":
["devxs","devm","loading","reporting"],"owns": {"databases": ["{{ prepdatabase }}","{{
proddatabase }}"],"schemas": ["{{ prepschemas }}","{{ prodschemas }}"],"tables": ["{{
preptables }}","{{ prodtables }}"],"privileges": {"databases": {"read": ["{{ prepdatabase
}}","{{ proddatabase }}"],"write": ["{{ prepdatabase }}","{{ proddatabase }}"],"schemas":
{"read": ["{{ prepschemas }}","{{ prodschema }}"],"write": ["{{ prepschemas }}","{{ prodschema
}}"],"tables": {"read": ["{{ preptables }}","{{ prodtables }}"],"write": ["{{ preptables
}}","{{ prodtables }}"]}}}}
```

Users

Default:

sh

```
USERTEMPLATE: {"{{ username }}": {"canlogin": true, "memberof": ["{{ username }}"]}}
```

Common: N/A. There are no other templates that we currently use for users

</details>

Automating